ТОО ”Колледж Хекслет”

Лабораторная работа №5

Подключение Prisma ORM к существующей базе shop_db

Выполнено студентом:

Дубинин Кирилл

Практическое задание

Раздел RO4_3. Подключение Prisma ORM к существующей базе shop_db.

Цель: Подключить Prisma ORM к существующему Express + PostgreSQL проекту, получить Prisma-модели из уже созданной базы и выполнить первые операции чтения через Prisma Client. В этой лабораторной работе не создаём новую базу и не удаляем существующие таблицы. Используем: • shop_db с таблицами: • users • user_profiles • orders • order_items • products • categories Полный CRUD через Prisma будет темой следующей лабораторной работы.

Условие: RO4_3/lesson5/ЛР-5.pdf

Требования задания

• использовать существующую shop_db;

• установить Prisma ORM;

• настроить DATABASE_URL;

• выполнить prisma db pull;

• получить модели существующих таблиц;

• использовать @map и @@map там, где это необходимо;

• выполнить prisma generate;

• создать один экземпляр PrismaClient;

• использовать PostgreSQL adapter;

• получить товары через findMany();

• получить один товар через findUnique();

• подключить Prisma к Product Service;

• сохранить существующий Controller;

• сохранить существующие REST endpoints;

• открыть данные через Prisma Studio;

• не удалять существующую PostgreSQL-базу.

Ход выполнения

Prisma 7 подключена к той же PostgreSQL через PrismaPg adapter. Introspection существующих таблиц сохранена в prisma/introspection.txt.

Модели приведены к Product, Category, User и другим именам с @map/@@map. Генерируется TypeScript Client, приложение запускается через tsx. Чтение переведено на findMany/findUnique; SQL-версия сохранена для сравнения. Baseline отмечается применённой для существующей схемы.

Проект: RO4_3/lesson5

Запуск: npm install; npm start. Для PostgreSQL параметры подключения задаются в .env по образцу .env.example. Порядок применения SQL и миграций приведён в README.md проекта.

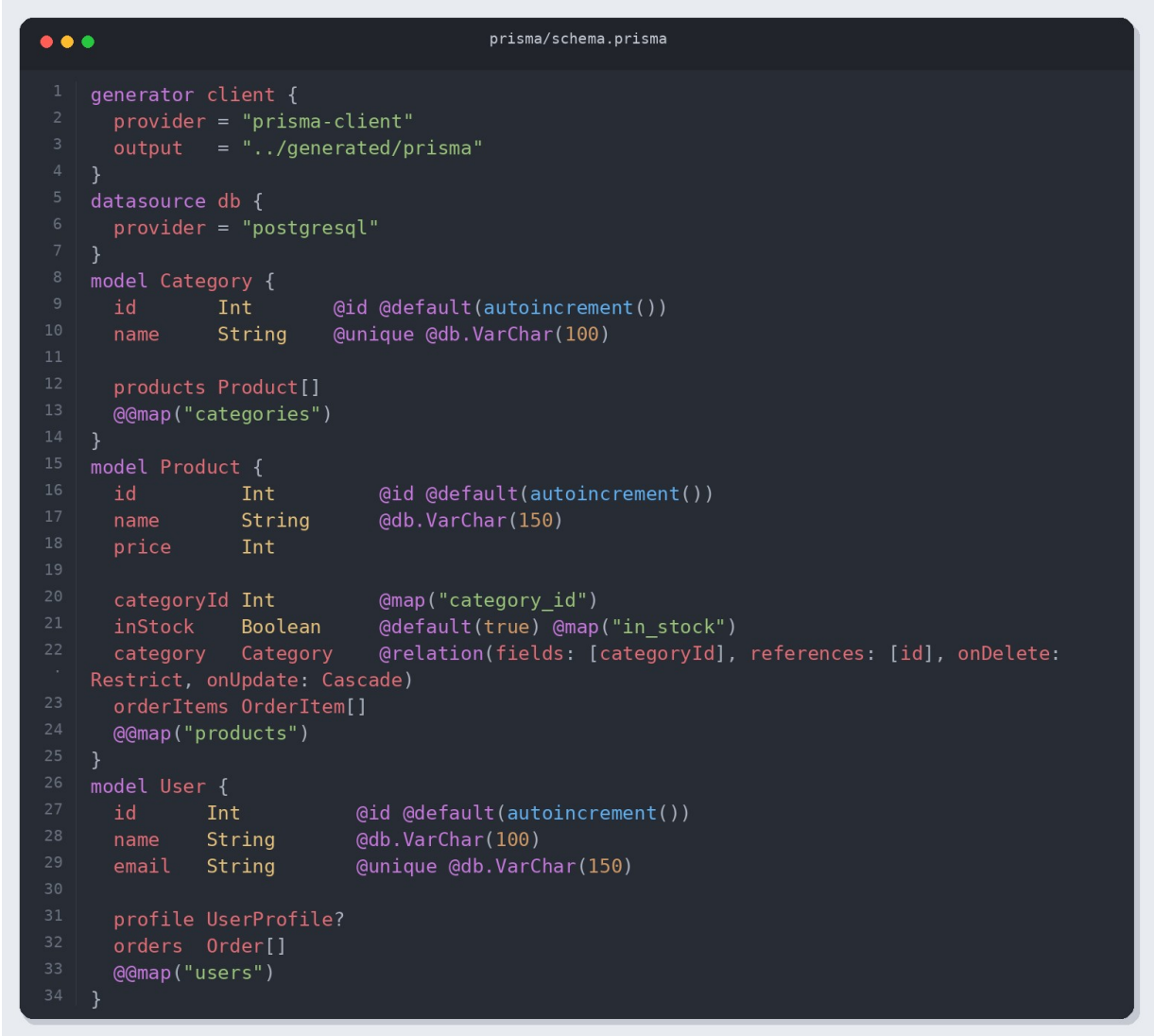
prisma.config.ts

prisma.config.ts

```ts
import 'dotenv/config';
import { defineConfig, env } from 'prisma/config';
export default defineConfig({
    schema: 'prisma/schema.prisma',
    migrations: { path: 'prisma/migrations', seed: 'tsx prisma/seed.js' },
    datasource: { url: env('DATABASE_URL') }
});
```

Листинг 1. prisma.config.ts

prisma/schema.prisma



Листинг 2. prisma/schema.prisma (часть 1)

prisma/schema.prisma

```
model UserProfile {
  userId  Int     @id @map("user_id")
  phone   String? @db.VarChar(30)
  address String? @db.VarChar(250)
  user    User    @relation(fields: [userId], references: [id], onDelete: Restrict,
onUpdate: Cascade)
  @@map("user_profiles")
}
model Order {
  id        Int         @id @default(autoincrement())
  userId    Int         @map("user_id")
  status    String      @default("new") @db.VarChar(30)
  createdAt DateTime    @default(now()) @map("created_at") @db.Timestamptz(6)
  user      User        @relation(fields: [userId], references: [id], onDelete:
Restrict, onUpdate: Cascade)
  items     OrderItem[]
  @@map("orders")
}
model OrderItem {
  orderId   Int     @map("order_id")
  productId Int     @map("product_id")
  quantity  Int
  price     Int
  order     Order   @relation(fields: [orderId], references: [id], onDelete: Restrict,
onUpdate: Cascade)
  product   Product @relation(fields: [productId], references: [id], onDelete: Restrict,
 onUpdate: Cascade)
  @@id([orderId, productId])
  @@map("order_items")
}
```

Листинг 3. prisma/schema.prisma (часть 2)

config/prisma.js

config/prisma.js

```
import './env.js';
import { PrismaPg } from '@prisma/adapter-pg';
import { PrismaClient } from '../generated/prisma/client.ts';
const adapter = new PrismaPg({ connectionString: process.env.DATABASE_URL });
export const prisma = new PrismaClient({ adapter });
```

Листинг 4. config/prisma.js

services/productService.js

services/productService.js

```
import { prisma } from '../config/prisma.js';
export async function getAllProducts(query = {}) {
    const where = {};
    if (query.category) where.category = { name: query.category };
    if (query.inStock !== undefined) where.inStock = query.inStock === 'true';
    if (query.minPrice !== undefined || query.maxPrice !== undefined) {
        where.price = {};
        if (query.minPrice !== undefined) where.price.gte = Number(query.minPrice);
        if (query.maxPrice !== undefined) where.price.lte = Number(query.maxPrice);
    }
    return prisma.product.findMany({ where, orderBy: { id: "asc" }, include: { category:
 true } });
}
export function findProductById(id) { return prisma.product.findUnique({ where: { id },
include: { category: true } }); }
export { addProduct, updateProductById, deleteProductById } from
'./productService.sql.js';
```

Листинг 5. services/productService.js

Проверка результата

Проверено: 05.11.2026 13:38 (Алматы). Node.js v26.12.1.

Приведены фактические результаты проверки. Полный журнал: verification.json.

• Реальный PostgreSQL

• HTTP endpoints

• CRUD сохраняется в PostgreSQL

• UNIQUE и FOREIGN KEY; категории перенесены без потери товаров

• Связи 1:1, 1:N, M:N; CHECK(quantity > 0); составной ключ

• Prisma Studio открыт в Chromium; таблица products содержит Keyboard

HTTP-запросы

GET /api/products → 200

GET /api/products/1 → 200

GET /api/products/999999 → 404

GET /api/products/abc → 400

GET /api/products?category=Accessories&inStock=true&minPrice=10000&maxPrice=50000 → 200

GET /api/products?minPrice=abc → 400

GET /api/products?inStock=maybe → 400

GET /api/products?minPrice=500&maxPrice=100 → 400

POST /api/products → 403

POST /api/products → 400

POST /api/products → 201

PATCH /api/products/6 → 200

DELETE /api/products/6 → 200

GET /api/products/6 → 404

POST /api/products → 409

GET /api/orders/1/items → 200

GET /api/orders/999999/items → 404

GET /api/orders/abc/items → 400

GET /api/unknown → 404

Примеры ответов

POST /api/products → 201


```
{
  "id": 6,
  "name": "Test SSD",
  "price": 65000,
  "categoryId": 1,
  "category": "Accessories",
  "inStock": true
}
```

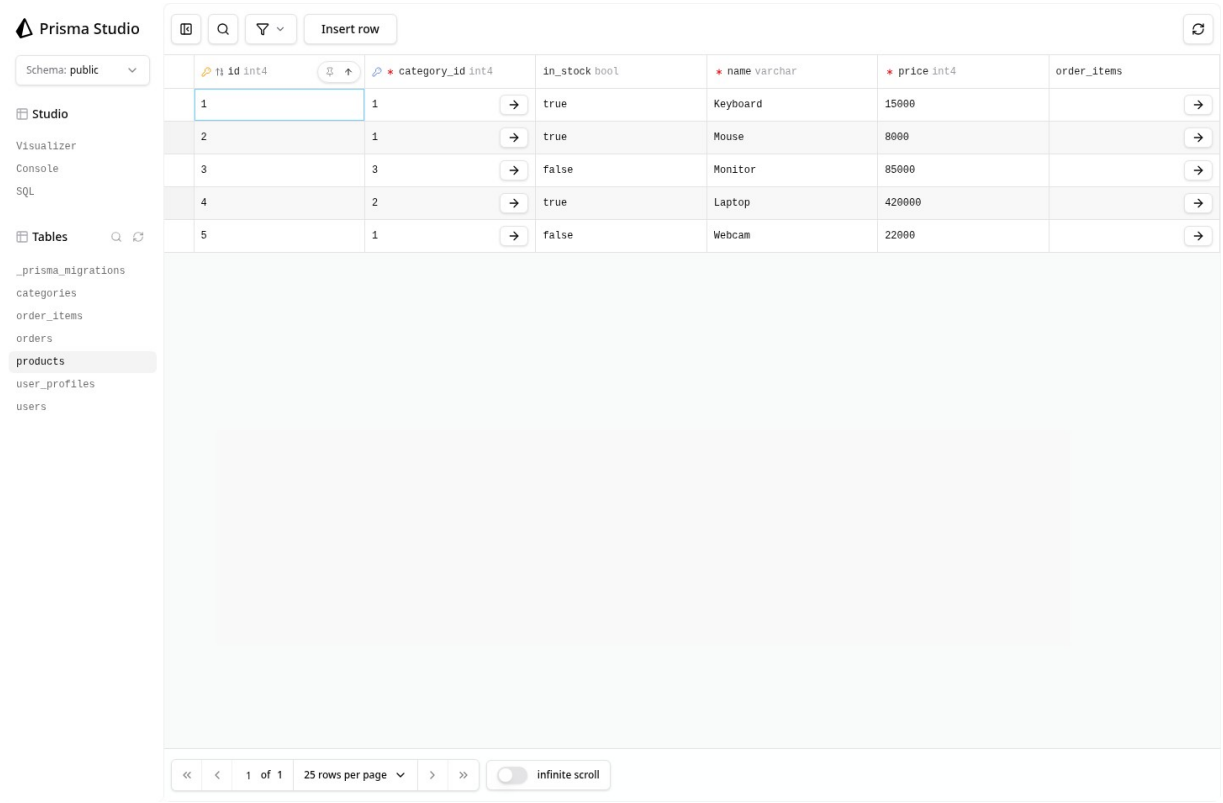

PATCH /api/products/6 → 200


```
{
  "id": 6,
  "name": "Test SSD",
  "price": 42000,
  "categoryId": 1,
  "category": "Accessories",
  "inStock": false
}
```


DELETE /api/products/6 → 200


```
{
  "id": 6,
  "name": "Test SSD",
  "price": 42000,
  "category_id": 1,
  "inStock": false
}
```

Проверка данных в Prisma Studio



Prisma Studio: таблицы и данные проверенной учебной базы.

Результат

Проверены обязательные операции и указанные ошибочные сценарии; результаты сохранены вместе с исходным кодом.